

第三次作业

作业发布时间：2024/05/19 星期一

本次作业要求如下：

1.截止日期：2024/05/30 周五晚 24:00

2.提交作业给本课程 QQ 群里的助教（XXX: QQ XXXX）

3.命名格式：附件和邮件命名统一为“第三次作业+学号+姓名”， PDF 格式；

4.注意：

(1) 选择、判断和填空只需要写答案，大题要求有详细过程，过程算分。

(2) 答案请用另一种颜色的笔回答，便于批改，否则视为无效答案。

(3) 大题的过程最好在纸上写了拍照，放到 word 里。

(4) 本次作业由 Part1 和 Part2 两部分，需要全部作答。

5.本次作业遇到问题请联系课程群里的助教。

Part1 存储器层次结构+虚拟内存

一. 填空题

1. 对于一个磁盘，其平均旋转速率是 15000 RPM，平均寻道时间是 4 ms，单个磁道上平均扇区数量是 800，则这个磁盘的平均访问时间是 6.005ms

2. 对于一个磁盘，其有两个扇片，10000 个柱面，每个磁道平均有 400 个扇区，而每个扇区平均有 512 个字节。那么这个磁盘的容量为 8.192GB

3. Cache 为 8 路的 2M 容量， $B=64$ ，则其 Cache 组的位数 $s =$ 12

4. 在现代计算机存储层次体系中，访问速度最快的是 寄存器

5. 若高速缓存的块大小为 B ($B > 8$) 字节，向量 v 的元素为 int，则对 v 的步长为 1 的应用的不命中率为 $4/B$

6. 某 CPU 使用 32 位虚拟地址和 4KB 大小的页时，需要 PTE 的数量是 2^{20}
(不考虑多级页表情况)

7. 缓存不命中的种类有 强制不命中、冲突不命中、容量不命中。

8. 虚拟页面的状态有 活跃、已缓存、未缓存共 3 种。

9. Linux 虚拟内存区域可以映射到普通文件和 匿名内存，这两种类型

的对象中的一种。

10. 虚拟内存发生缺页时，缺页中断是由 ____内存管理单元 MMU____ 触发的。

11. Linux 缺页异常可能的几种原因分别是 __访问尚未映射到物理内存的有效虚拟地址、首次访问按需分配的页、访问已换出的页、访问不存在的虚拟地址____。

二. 分析题

1. 假设 1MB 的文件由平均大小为 512 bytes 的逻辑块组成。磁盘的转速为 10000 RPM，平均寻道时间是 5 ms，每个磁道上的平均扇区数量为 1000，扇面大小为 4，单个扇区大小为 512 bytes。那么访问这个文件最好的时间和最坏的时间分别是多少？

答：

最好：

当文件连续存储在一个柱面内时，访问只需一次寻道、一次旋转延迟和连续传输所有数据：

寻道时间：5 ms（平均）。

平均旋转延迟：3 ms。

传输时间：

文件大小 1,048,576 字节。

传输时间 = $1,048,576 / 85,333,333 = 12.286$ ms。

总时间 = 寻道时间 + 平均旋转延迟 + 传输时间 = 20.286 ms。

最坏：

当文件扇区随机分散在磁盘上时，每个扇区访问需要独立的寻道、旋转延迟和传输：

每个扇区访问时间：

寻道时间：5 ms（平均）。

平均旋转延迟：3 ms。

传输时间：0.006 ms（可忽略，但保留）。

总 per-sector 时间 $\approx 5 + 3 + 0.006 = 8.0065 + 3 + 0.006 = 8.006$ ms。

总扇区数：2,048。

总时间 = $2,048 \times 8.006 = 16,396.2882, 048 \times 8.006 = 16,396.288$ ms。

2. 请解释什么是直接内存访问（DMA）。并简述 CPU 发起磁盘读的三条存储指令。

答：

直接内存访问（DMA）是计算机系统的一项功能，它允许硬件子系统（如磁盘控制器、网络接口卡等）直接访问主系统内存（RAM），而无需中央处理器（CPU）的干预。在传统的 I/O 操作中，CPU 需要参与每个字节的数据传输，从 I/O 设备读取数据到 CPU 寄存器，再从寄存器写入内存，或反之。这种方式会占用 CPU 大量时间，降低系统吞吐量。DMA 通过将数据移动操作从 CPU 卸载到专门的 DMA 控制器来解决这一问题。DMA 控制器是一个独立的硬件单元，它能够自

主地在 I/O 设备和内存之间传输数据。这种机制允许 CPU 和 DMA 控制器并发工作，实现真正的多任务处理行为

当 CPU 发起磁盘读操作时，通常需要通过向 I/O 控制器发送存储指令来启动 DMA。以下是关键的三条指令（写入特定寄存器）：

1. 写磁盘块号（或磁盘地址）到数据寄存器
 - a) 指令示例：OUT 寄存器_地址, 块号
 - b) 说明：指定要读取的磁盘块号（或扇区号），告诉磁盘控制器从哪个位置读取数据。
2. 写内存缓冲区地址到内存地址寄存器
 - a) 指令示例：OUT 寄存器_地址, 内存地址
 - b) 说明：指定读入的数据在内存中的存放位置。
3. 写命令到命令寄存器以启动 DMA 传输
 - a) 指令示例：OUT 命令寄存器, 启动命令（如 READ）
 - b) 说明：通知磁盘控制器开始 DMA 传输，数据会从磁盘直接写入内存，无需 CPU 再干预。

3. 对于一个机器而言，有如下的假设，内存是字节寻址，并且内存访问是 1 字节的字。地址宽度是 13 位，其高速缓存是 2 路组相联的，块大小是 4 字节，一共有 8 个组。高速缓存的具体内容如图所示。

组索引	行0						行1					
	标记位	有效位	字节0	字节1	字节2	字节3	标记位	有效位	字节0	字节1	字节2	字节3
0	09	1	86	30	3F	10	00	0	—	—	—	—
1	45	1	60	4F	E0	23	38	1	00	BC	0B	37
2	EB	0	—	—	—	—	0B	0	—	—	—	—
3	06	0	—	—	—	—	32	1	12	08	7B	AD
4	C7	1	06	78	07	C5	05	1	40	67	C2	3B
5	71	1	0B	DE	18	4B	6E	0	—	—	—	—
6	91	1	A0	B7	26	2D	F0	0	—	—	—	—
7	46	0	—	—	—	—	DE	1	12	C0	88	37

- a) 对于该地址格式进行划分，划分出 tag 位，组索引位和块内偏移的区间。
- b) 对于地址 0x0E34，指出其对应的 tag 位，组索引位和块内偏移的值，并说明高速缓存是否命中，如果命中，写出对应的字节（用 16 进制表示）。
- c) 对于地址 0x0DD5，指出其对应的 tag 位，组索引位和块内偏移的值，并说明高速缓存是否命中，如果命中，写出对应的字节（用 16 进制表示）。

答：

a)

offset 2 位 [1:0] 一个块内有 4 个字节

index 3 位 [4:2] 共 8 个组

tag 8 位 [12:5] 总共 13 位，剩下 8 位

b) 由于 0x0E34=0111000110100，

offset ([1:0]): 00 → 十进制：0

index ([4:2]): 101 → 十进制: 5

tag ([12:5]): 0111 0001 → 十六进制: 0x71

到 Cache 中 组索引 5 查看行 0 可知, 其有效位为 1, tag=0x71 → 匹配
则说明高速缓存命中, 所对应的字节为 0x0B

c) 由于 0x0DD5=011011101 0101, 则:

offset ([1:0]): 01 → 十进制: 1

index ([4:2]): 101 → 十进制: 5

tag ([12:5]): 0110 1110 → 十六进制: 0x6E

到 Cache 中 组索引 5 查看行 0 可知, 其有效位为 1, tag=0x71≠0x6E → 不匹配; 查看行 1 可知, 其有效位为 0 → 不匹配。说明高速缓存不命中。

4. 对于一个直接映射的高速缓存系统, 假设其大小是 256 字节, 块大小是 16 字节, 现在定义三个操作, L 为装载操作, S 为数据存储操作, M 为数据更改操作。L 和 S 最多引发一次缓存 miss, 而 M 操作可以看作是对于同一个地址先进行了 L 操作, 之后进行了 S 操作。分析下面的操作序列, 对于高速缓存的命中和淘汰情况。(假设高速缓存最开始是空的)

L 10, 1

M 20, 1

L 22, 1

S 18, 1

L 110, 1

L 210, 1

M 12, 1

说明: L 10, 1 表示对于地址 0x10 位置, 进行了一个字节的装载操作。

答:

1. L 10, 1

- a) 地址 0x10 属于内存块 0x10-0x1F。
- b) Cache 行索引 = $(0x10 / 0x10) \% 16 = 1 \% 16 = 1$ 。
- c) 结果: 不命中 (Cache 为空, 强制不命中)。
- d) 操作: 将内存块 0x10-0x1F 加载到 Cache 行 1。

2. M 20, 1

- a) M 操作视为 L 后跟 S:
- b) L 20, 1 (加载地址 0x20 的 1 字节)
 - i. 地址 0x20 属于内存块 0x20-0x2F。
 - ii. Cache 行索引 = $(0x20 / 0x10) \% 16 = 2 \% 16 = 2$ 。
 - iii. 结果: 不命中 (Cache 行 2 为空)。
 - iv. 操作: 将内存块 0x20-0x2F 加载到 Cache 行 2。
- c) S 20, 1 (存储地址 0x20 的 1 字节)
 - i. 地址 0x20 仍在内存块 0x20-0x2F, Cache 行索引为 2。
 - ii. 结果: 命中 (该块已在 Cache 行 2 中)。
 - iii. 操作: 更新 Cache 行 2 中 0x20 处的数据。

3. L 22, 1

- a) 地址 0x22 属于内存块 0x20-0x2F。
 - b) Cache 行索引 = 2。
 - c) 结果：命中 (该块已在 Cache 行 2 中)。
 - d) 操作：从 Cache 行 2 访问数据。
4. S 18, 1
- a) 地址 0x18 属于内存块 0x10-0x1F。
 - b) Cache 行索引 = 1。
 - c) 结果：命中 (该块已在 Cache 行 1 中)。
 - d) 操作：更新 Cache 行 1 中 0x18 处的数据。
5. L 110, 1
- a) 地址 0x110 属于内存块 0x110-0x11F (十进制 272-287)。
 - b) Cache 行索引 = $(0x110 / 0x10) \% 16 = 17 \% 16 = 1$ 。
 - c) 结果：不命中 (Cache 行 1 当前存放的是 0x10-0x1F 的数据，标记不匹配)。
 - d) 操作：淘汰 Cache 行 1 中的 0x10-0x1F 块，加载内存块 0x110-0x11F 到 Cache 行 1。
6. L 210, 1
- a) 地址 0x210 属于内存块 0x210-0x21F (十进制 528-543)。
 - b) Cache 行索引 = $(0x210 / 0x10) \% 16 = 33 \% 16 = 1$ 。
 - c) 结果：不命中 (Cache 行 1 当前存放的是 0x110-0x11F 的数据，标记不匹配)。
 - d) 操作：淘汰 Cache 行 1 中的 0x110-0x11F 块，加载内存块 0x210-0x21F 到 Cache 行 1。
7. M 12, 1
- a) M 操作视为 L 后跟 S：
 - b) L 12, 1
 - i. 地址 0x12 属于内存块 0x10-0x1F。
 - ii. Cache 行索引 = $(0x12 / 0x10) \% 16 = 1 \% 16 = 1$ 。
 - iii. 结果：不命中 (Cache 行 1 当前存放的是 0x210-0x21F 的数据，标记不匹配)。
 - iv. 操作：淘汰 Cache 行 1 中的 0x210-0x21F 块，加载内存块 0x10-0x1F 到 Cache 行 1。
 - c) S 12, 1
 - i. 地址 0x12 仍在内存块 0x10-0x1F，Cache 行索引为 1。
 - ii. 结果：命中 (该块已在 Cache 行 1 中)。
 - iii. 操作：更新 Cache 行 1 中 0x12 处的数据。

共有 5 次 miss，4 次 hit；有 3 次淘汰发生(由于不同块映射到同一缓存行)，显示了直接映射缓存的缺点：冲突容易导致频繁替换，即使数据仍然有效。

4. 对于著名的存储器山形状图，分析读吞吐量产生如此变化趋势的原因，说明几条山脊和斜坡出现的原因。

答：

存储器山的形状是由存储器层次结构（寄存器、L1 Cache、L2 Cache、L3 Cache、主存、磁盘等）及其局部性原理共同决定的。程序倾向于访问最近使用过的数据

（时间局部性）和与最近使用过的数据相邻的数据（空间局部性）。Cache 通过将主存中的数据块复制到更小、更快的存储器中来利用这些局部性。当数据在 Cache 中命中时，访问速度快；当不命中时，需要从更慢的下一级存储器中获取数据，导致性能下降。

山脊是图中水平的、高吞吐量的平台区域，它们主要由时间局部性和 Cache 容量决定。当程序的工作集大小小于或等于某一 Cache 级别的容量时，所有活跃数据都可以完全驻留在该 Cache 中。这意味着对这些数据的后续访问将大部分是 Cache 命中，从而获得该 Cache 级别所能提供的最高吞吐量。

斜坡是图中吞吐量显著下降的区域，它们主要由容量不命中和空间局部性丧失决定。当工作集大小超过当前 Cache 级别的容量时，Cache 无法容纳所有活跃数据。程序必须频繁地从下一级更慢的存储器中获取数据，导致大量的容量不命中。每次容量不命中都会带来显著的延迟，从而使吞吐量急剧下降，形成陡峭的斜坡。

5. 在一台具有块大小 16 字节（B=16）、整个大小为 1024 字节的直接映射数据缓存的机器上测量如下代码的高速缓存性能：

假设：

- `sizeof(int) = 4`。
- `grid` 从内存地址 0 开始。
- 这个高速缓存开始时是空的。
- 唯一的内存访问是对数组 `grid` 的元素的访问，变量 `i`、`j`、`total_x` 和 `total_y` 存放在寄存器中。
- 数据结构定义

```
struct position {  
    int x;  
    int y;  
};
```

```
struct position grid[16][16];
```

```
int total_x = 0, total_y = 0;  
int i, j;
```

```
    A. Test 1  
for (i = 0; i < 16; i++){  
    for(j = 0; j < 16; j++){  
        total_x += grid[i][j].x;  
    }  
}  
  
for (i = 0; i < 16; i++){  
    for(j = 0; j < 16; j++){
```

```

        total_y += grid[i][j].y;
    }
}

```

1. 读总数是多少?
2. 缓存不命中的读总数是多少?
3. 不命中率是多少?

答:

1. 第一个循环: $16(i) * 16(j) * 1(\text{访问 } x) = 256$ 次读。

第二个循环: $16(i) * 16(j) * 1(\text{访问 } y) = 256$ 次读。

总读数 = $256 + 256 = 512$ 次。

2. grid 数组总大小为 2048 字节, 包含 $2048 / 16 = 128$ 个内存块。

Cache 大小为 1024 字节, 包含 $1024 / 16 = 64$ 个 Cache 块。

由于是直接映射 Cache, 内存块 M 映射到 Cache 块 $M \% 64$ 。

grid 数组的内存块编号从 0 到 127

第一个循环 ($\text{total_x} += \text{grid}[i][j].x$):

每次访问 $\text{grid}[i][j].x$ 时, 会加载包含 $\text{grid}[i][j]$ 和 $\text{grid}[i][j+1]$ 的 16 字节块。

每行 i 有 16 个 struct position 元素, 需要访问 $16 / 2 = 8$ 个 Cache 块。

对于 $i=0$ 到 $i=7$ (前 8 行):

访问内存块 0 到 63。这些块会填充 Cache 块 0 到 63。

每行有 8 个不命中 (强制不命中)。

总不命中数 = $8 \text{ 行} * 8 \text{ 不命中/行} = 64$ 次。

对于 $i=8$ 到 $i=15$ (后 8 行):

访问内存块 64 到 127。

内存块 64 映射到 Cache 块 $64 \% 64 = 0$, 会淘汰 grid 所在的块。

内存块 65 映射到 Cache 块 $65 \% 64 = 1$, 会淘汰 grid 所在的块。

...

所有这些访问都会导致不命中 (冲突不命中), 因为它们映射到已被前 8 行占据的 Cache 块, 并将其淘汰。

总不命中数 = $8 \text{ 行} * 8 \text{ 不命中/行} = 64$ 次。

第一个循环总不命中数 = $64 + 64 = 128$ 次。

在第一个循环结束时, Cache 中包含 grid 到 grid 的数据。

第二个循环 ($\text{total_y} += \text{grid}[i][j].y$):

这个循环再次从 $i=0$ 开始访问。

对于 $i=0$ 到 $i=7$:

这些数据在第一个循环的 $i=0$ 到 $i=7$ 阶段被加载, 但随后被 $i=8$ 到 $i=15$ 阶段的数据淘汰。

因此, 再次访问这些块时, 它们不在 Cache 中, 导致不命中 (冲突不命中)。

每行有 8 个不命中。

总不命中数 = $8 \text{ 行} * 8 \text{ 不命中/行} = 64$ 次。

对于 $i=8$ 到 $i=15$:

这些数据在第一个循环结束时存在于 Cache 中。

在第二个循环中， $i=0$ 到 $i=7$ 的访问又会淘汰掉 $i=8$ 到 $i=15$ 的部分数据。

当访问 $i=8$ 到 $i=15$ 时，这些块又被淘汰了，导致不命中（冲突不命中）。

总不命中数 = 8 行 * 8 不命中/行 = 64 次。

第二个循环总不命中数 = $64 + 64 = 128$ 次。

缓存不命中的读总数： $128 + 128 = 256$

3. 不命中率： $256 / 512 = 0.5 = 50\%$

B. Test 2

```
for (i = 0; i < 16; i++){
    for(j = 0; j < 16; j++){
        total_x += grid[i][j].x;
        total_y += grid[i][j].y;
    }
}
```

1. 读总数是多少？
2. 缓存不命中的读总数是多少？
3. 不命中率是多少？
4. 如果高速缓存有两倍大，那么不命中率是多少？

答：

1. $16(i) * 16(j) * 2$ (访问 x 和 y) = 512 次读

总读数 = 512 次

2. 在内层循环中， $grid[i][j].x$ 和 $grid[i][j].y$ 被连续访问。由于它们属于同一个 `struct position` 元素（8 字节），并且一个 Cache 块（16 字节）可以容纳两个 `struct position` 元素，这意味着当 $grid[i][j].x$ 被访问导致 Cache 不命中时，整个 16 字节的块（包含 $grid[i][j]$ 和 $grid[i][j+1]$ ）会被加载。

此时， $grid[i][j].y$ 的访问将是命中。

$grid[i][j+1].x$ 和 $grid[i][j+1].y$ 的访问也是命中，因为它们都在同一个已加载的块中。

每加载一个 16 字节的 Cache 块（1 次不命中），可以满足 4 次 `int` 访问（ $grid[i][j].x$, $grid[i][j].y$, $grid[i][j+1].x$, $grid[i][j+1].y$ ）。

每行 i 有 16 个 `struct position` 元素，需要 8 个 Cache 块。

总共有 16 行，所以总共访问 $16 * 8 = 128$ 个不同的内存块。

由于 Cache 容量为 64 块，`grid` 数组（128 块）大于 Cache，因此会发生冲突不命中。

对于 $i=0$ 到 $i=7$ (前 8 行)：

访问内存块 0 到 63。这些都是强制不命中。

总不命中数 = 8 行 * 8 不命中/行 = 64 次。

对于 $i=8$ 到 $i=15$ (后 8 行)：

访问内存块 64 到 127。这些块映射到 Cache 块 0 到 63，会淘汰掉前 8 行的数据。

都是冲突不命中。总不命中数 = 8 行 * 8 不命中/行 = 64 次。

总不命中数 = 64 + 64 = 128 次。

3. 不命中率: $128 / 512 = 0.25 = 25\%$

4. 如果高速缓存有两倍大 (2048 字节):

新的 Cache 大小 = 2048 字节。

Cache 块数 = $2048 / 16 = 128$ 块。

现在 Cache 的块数 (128 块) 与 grid 数组的总内存块数 (128 块) 相同。

这意味着整个 grid 数组可以完全放入 Cache 中。一旦一个内存块被加载到 Cache 中, 它就不会因为与其他 grid 数组块的冲突而被淘汰。

所有不命中都将是强制不命中 (首次访问)。

总共需要加载 128 个唯一的内存块。

总读数: 512 次。

缓存不命中的读总数: 128 次。

不命中率: $128 / 512 = 0.25 = 25\%$

6. 对于一个地址, 高速缓存通常使用地址的中间部分作为组索引, 为什么不用高位地址作为组索引?

答: 如果将地址的高位作为组索引, 那么内存中大片连续的地址空间 (例如, 一个大型数组或数据结构) 将映射到 Cache 中的同一个组。这会导致严重的冲突不命中问题, 尤其是在直接映射 Cache 或低相联度的组相联 Cache 中。即使 Cache 的总容量足够大, 可以容纳程序的工作集, 但由于多个活跃的内存块都映射到同一个 Cache 组, 它们会不断地相互淘汰。

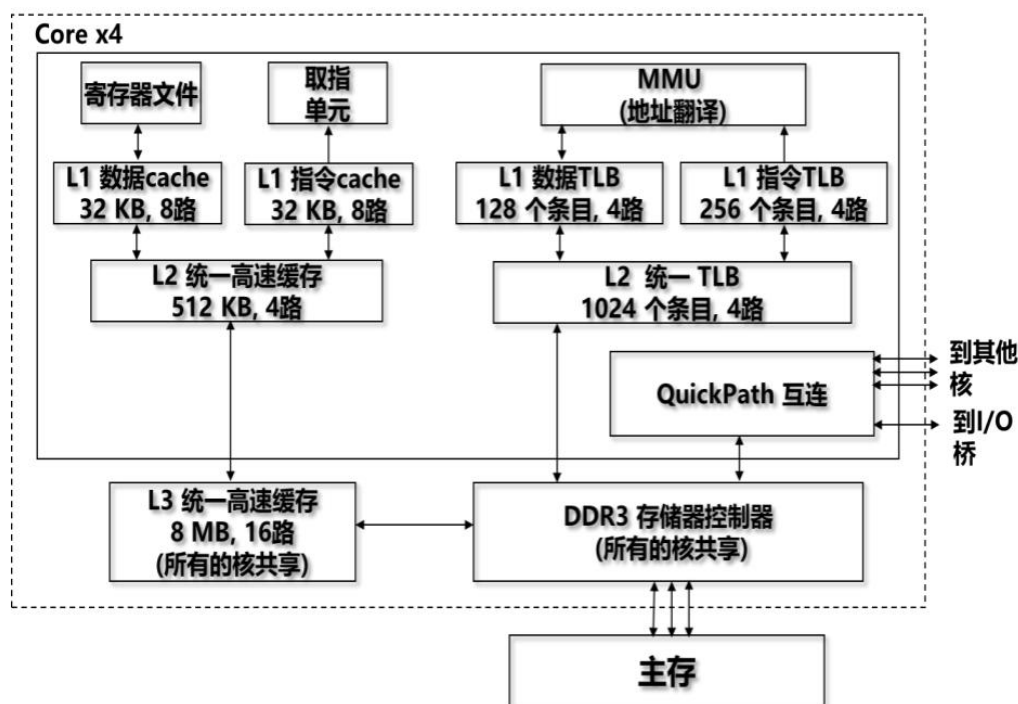
将地址的中间部分作为组索引, 通常紧邻块内偏移位, 可以有效地缓解冲突不命中问题, 并更好地利用空间局部性。这种策略确保了内存中物理上相邻的块通常会映射到 Cache 中不同的组。有助于将内存访问均匀地分布到 Cache 的各个组中, 从而提高了 Cache 的整体利用率。

7. 下图展示了一个虚拟地址的访存过程, 每个步骤都用不同的字母表示。请针对下面不同的情况, 用字母序列表示不同情况下的执行流程。

(1) TLB 命中, 缓存物理地址命中。

(2) TLB 不命中, 缓存页表命中, 缓存物理地址命中。

(3) TLB 不命中, 缓存页表不命中, 缓存物理地址不命中。



(1) 虚拟地址的 VPN 占多少位，一级页表占多少项？L1 数据 TLB 的组索引位数 TLBI 占多少位？

(2) L1 数据 Cache 有多少组，相应的 Tag 位，组索引位和块内偏移位分别是多少？

(3) 对于某指令，其访问的虚拟地址为 0x804849B，则该地址对应的 VPO 为多少？对应的 L1 TLBI 位为多少？（用 16 进制表示）

(4) 对于某指令，其访问的物理地址为 0x804849B，则该地址访问 L1 Cache 时，CT 位为多少？CO 位为多少？（用 16 进制表示）

答：

(1) 页大小为 4KB，页内偏移为 12 位。

对应的 VPN 为 36 位。四级页表结构，一级页表占 9 位，一共 512 项。

L1 数据 TLB 一共 128 个条目，4 路，共有 32 组，TLBI 占 5 位。

(2) L1 数据 Cache 大小 32KB，8 路，Cache 块大小为 64B，一共有 64 组。

组索引位为 6 位，块内偏移位为 6 位。

物理地址为 52 位，其余的 40 位为 Tag 位。

(3) L1 指令 TLB 一共 256 个条目，4 路，共有 64 组，TLBI 为 6 位。

虚拟地址为 0x804849B=10000000010010000100 1001 1011，VPO 为 0x49B，TLBI 位为 001000=0x08

(4) L1 指令 Cache 的 Tag 位为 40 位，组索引位为 6 位，块内偏移位为 6 位。

物理地址 0x804849B=100000000100 1000 0100 1001 1011，CT 为 0x8048，CO 为 0x1B

一. 选择题

12. 链接时两个文件同名的弱符号，以（ **B** ）为基准

- A. 连接时先出现的
- B. 连接时后出现的
- C. 任一个
- D. 链接报错

13. 链接时两个同名的强符号，以哪种方式处理？（ **D** ）

- A. 链接时先出现的符号为准
- B. 链接时后出现的符号为准
- C. 任一个符号为准
- D. 链接报错

14. 以下关于程序中链接“符号”的陈述，错误的是（ **B** ）

- A. 赋初值的非静态全局变量是全局强符号
- B. 赋初值的静态全局变量是全局强符号
- C. 未赋初值的非静态全局变量是全局弱符号
- D. 未赋初值的静态全局变量是本地符号

15. C 源文件 m1.c 和 m2.c 的代码分别如下所示，编译链接生成可执行文件后执行，结果最可能为（ **C** ）

\$ gcc -o a.out m2.c m1.c ; ./a.out

0x1083020 ; _____ ; _____

A. 0x1083018, 0x108301c B. 0x1083028, 0x1083024

C. 0x1083024, 0x1083028 D. 0x108301c, 0x1083018

<pre>// m1.c #include <stdio.h> int a1 ; int a2 = 2 ; extern int a4 ; void hello() { printf("%p;", &a1); printf("%p;", &a2); printf("%p\n", &a4); }</pre>	<pre>//m2.c int a4 = 10 ; int main() { extern void hello() ; hello() ; return 0 ; }</pre>
---	---

16. 对于以下一段代码，可能的输出为：（ **A** ）

```

int count = 0;
int pid = fork();
if (pid == 0){
    printf("count = %d\n",--count);
}
else{
    printf("count = %d\n",++count);
}
printf("count = %d\n",++count);

```

A.1 2 -1 0

B.0 0 -1 1

C.1 -1 0 0

D.0 -1 1 2

17. Linux 进程终止的原因可能是(**D**)

A.收到一个信号

B.从主程序返回

C.执行 exit 函数

D.以上都是

18. 下列函数中属于系统调用且调用一次，从不返回的是(**B**)

A.fork

B.execve

C.setjmp

D.longjmp

二. 填空题

1. C 语句中的全局变量，在__**链接**__阶段被定位到一个确定的内存地址。
2. 子程序运行结束会向父进程发送__**SIGCHLD**__信号。
3. 向指定进程发送信号的 linux 命令是__**kill**__。
4. Unix 内核通过三个表项__**文件描述符表**__、__**文件表**__、__**inode 表**__表示打开文件。
5. Unix 内核通过调用函数__**dup2**__实现 I/O 重定向。

三. 分析题

8. 请阅读以下程序，然后回答问题（假设程序中的函数调用都可以正确执行）：

```

int main() {
    printf("A\n");
    if (fork() == 0) {
        printf("B\n");
    }else {
        printf("C\n");
        A
    }
    printf("D\n"); exit(0);
}

```

(1) 如果程序中的 A 位置的代码为空，列出所有可能的输出结果：

(2) 如果程序中的 A 位置的代码为:

```
waitpid(-1, NULL, 0);
```

列出所有可能的输出结果:

(3) 如果程序中的 A 位置的代码为:

```
printf("E\n");
```

列出所有可能的输出结果:

答:

(1)

- 父进程先执行: 打印 "A"、"C"、"D" (父进程的 D), 然后子进程执行: 打印 "B"、"D" (子进程的 D): ACDBD
- 父进程打印 "A"、"C" 后, 子进程执行打印 "B", 然后父进程打印 "D" (父进程的 D), 子进程打印 "D" (子进程的 D); 或子进程打印 "B" 后立即打印 "D" (子进程的 D), 然后父进程打印 "D" (父进程的 D)。由于两个 "D" 内容相同, 序列相同: ACBDD
- 子进程在父进程打印 "C" 前执行: 打印 "A" 后, 子进程先打印 "B", 然后父进程打印 "C", 最后两个进程打印 "D" (顺序不定, 但序列相同): ABCDD
- 子进程先执行完成: 打印 "A" 后, 子进程打印 "B"、"D" (子进程的 D), 然后父进程打印 "C"、"D" (父进程的 D): ABD CD

(2) waitpid 使父进程阻塞, 直到子进程退出。子进程必须先完成执行, 父进程才能继续。

- 子进程先执行: 打印 "A" 后, 子进程打印 "B"、"D" (子进程的 D) 并退出; 然后父进程打印 "C", waitpid 立即返回 (因子进程已退出), 再打印 "D" (父进程的 D): ABD CD
- 父进程先执行部分代码: 打印 "A"、"C" 后调用 waitpid 阻塞; 然后子进程执行: 打印 "B"、"D" (子进程的 D) 并退出; 父进程的 waitpid 返回, 打印 "D" (父进程的 D): ACBDD

(3) 父进程在 else 分支中打印 "C" 后, 额外打印 "E", 然后打印 "D"。

满足顺序约束: C 在 E 前, E 在父进程的 D 前, B 在子进程的 D 前:

- 子进程先执行完成: 打印 "A" 后, 子进程打印 "B"、"D" (子进程的 D); 然后父进程打印 "C"、"E"、"D" (父进程的 D): ABD CED
- 子进程在父进程打印 "E" 前执行: 打印 "A" 后, 子进程打印 "B"; 父进程打印 "C"; 子进程打印 "D" (子进程的 D); 父进程打印 "E"、"D" (父进程的 D): ABCDED
- 子进程在父进程打印 "D" 前执行: 打印 "A" 后, 子进程打印 "B"; 父进程打印 "C"、"E"; 然后两个进程打印 "D" (顺序不定, 但序列相同): ABCEDD
- 父进程打印 "C" 后, 子进程执行: 打印 "A"、"C" 后, 子进程打印 "B"; 子进程打印 "D" (子进程的 D); 父进程打印 "E"、"D" (父进程的 D): ACBDED
- 父进程打印 "C" 后, 子进程打印 "B", 然后父进程打印 "E"; 最后两个进程打印 "D" (顺序不定, 但序列相同): ACBEDD
- 父进程打印 "C"、"E" 后, 子进程执行: 打印 "B"; 然后两个进程打印 "D" (顺序不定, 但序列相同): ACEBDD

- 父进程先执行部分代码：打印 "A"、"C"、"E"、"D"（父进程的 D）；然后子进程执行：打印 "B"、"D"（子进程的 D）：ACEDBD

9. 假设一个 C 语言程序有两个源文件: main.c 和 proc1.c, 它们的内容如下图所示

```
1 #include <stdio.h>
2 unsigned x=257;
3 short y, z=2;
4 void proc1(void);
5 void main()
6 {
7     proc1();
8     printf("x=%u,z=%d\n", x, z);
9     return 0;
10 }
```

a) main.c 文件

```
1 double x;
2
3 void proc1()
4 {
5     x=-1.5;
6 }
```

b) proc1.c 文件

回答下列问题:

- 在上述两个文件中出现的符号哪些是强符号? 哪些是弱符号? 各变量的存储空间分配在哪个节中? 各占几个字节?
- 程序执行后打印的结果是什么? 请分别画出执行第 7 行的 proc1()函数调用前、后, 在地址&x 和&z 中存放的内容。
- 若 main.c 的第 3 行改为“short y = 1, z = 2;”,结果又会怎样?
- 修改文件 proc1, 使得 main.c 能输出正确的结果(即 x = 257, z = 2)。要求修改时不能改变任何变量的数据类型和名字。

答:

a) 强符号: main.c 中的 x、y、main 函数; proc1.c 中的 proc1 函数。

弱符号: proc1.c 中的 x

存储分配:

x (main.c): .data 节, 4 字节。

y (main.c): .data 节, 2 字节。

x (proc1.c): 弱符号, 被覆盖, 不分配。

函数: .text 节

b) main.c 中的 x 是 unsigned x=257; 而 proc1.c 中的 x 是 double x; 名字相同但类型不同, 链接器会按强符号优先, 最终 x 取自 main.c, 其类型为 unsigned。

x 的值被修改为(unsigned)-1= 4294967295。程序运行结果为: x=4294967295,z=2

c) 此时 y 变成了强符号。不影响程序行为, y 没有参与任何运算或输出, 改成强符号不影响执行结果。仍然输出: x=4294967295,z=2

d) 将 x=-1.5; 删除

10. 两个 C 语言程序 main.c、test.c 如下所示

<pre> /* main.c */ #include <stdio.h> int a[4]={-1,-2,2, 3}; extern int val; int sum(); int main(int argc, char * argv[]) { val=sum(); printf("sum=%d\n",val); } </pre>	<pre> /* test.c */ extern int a[]; int val=0; int sum() { int i; for (i=0; i<4; i++) val += a[i]; return val; } </pre>
--	---

用如下两条指令编译、链接，生成可执行程序 test:

```
gcc -m64 -no-pie -fno-PIC -c test.c main.c
```

```
gcc -m64 -no-pie -fno-PIC -o test test.o main.o
```

运行指令 `objdump -dxs main.o` 输出的部分内容如下:

Contents of section .data:

```
0000 ffffffff feffffff 02000000 03000000 .....
```

Contents of section .rodata:

```
0000 73756d3d 25640a00          sum=%d..
```

...

Disassembly of section .text:

0000000000000000 <main>:

```

0: 55                push    %rbp
1: 48 89 e5          mov     %rsp,%rbp
4: 48 83 ec 10       sub     $0x10,%rsp
8: 89 7d fc          mov     %edi,-0x4(%rbp)
b: 48 89 75 f0       mov     %rsi,-0x10(%rbp)
f: b8 00 00 00 00    mov     $0x0,%eax
14: e8 00 00 00 00    callq   19 <main+0x19>
      15: R_X86_64_PC32 sum-0x4
19: 89 05 00 00 00 00 mov     %eax,0x0(%rip) # 1f <main+0x1f>
      1b: R_X86_64_PC32 val-0x4
1f: 8b 05 00 00 00 00 mov     0x0(%rip),%eax # 25 <main+0x25>
      21: R_X86_64_PC32 val-0x4
25: 89 c6            mov     %eax,%esi
27: bf 00 00 00 00    mov     $0x0,%edi
      28: R_X86_64_32 .rodata
2c: b8 00 00 00 00    mov     $0x0,%eax
31: e8 00 00 00 00    callq   36 <main+0x36>
      32: R_X86_64_PC32 printf-0x4
36: b8 00 00 00 00    mov     $0x0,%eax
3b: c9              leaveq
3c: c3              retq

```

objdump -dxs test 输出的部分内容如下 (■是没有显示的隐藏内容):

SYMBOL TABLE:

```
0000000000400400 l    d  .text 0000000000000000      .text
00000000004005e0 l    d  .rodata 0000000000000000      .rodata
0000000000601020 l    d  .data0000000000000000      .data
0000000000601040 l    d  .bss 0000000000000000      .bss
0000000000000000      F *UND* 0000000000000000      printf@@GLIBC_2.2.5
0000000000601044 g    O .bss 0000000000000004      val
0000000000601030 g    O .data 0000000000000010      a
00000000004004e7 g    F .text 0000000000000039      sum
0000000000400400 g    F .text 000000000000002b      _start
0000000000400520 g    F .text 000000000000003d      main
```

Contents of section .rodata:

```
4005e0 01000200 73756d3d 25640a00      ....sum=%d..
```

...

Contents of section .data:

```
601020 00000000 00000000 00000000 00000000      .....
601030 ffffffff feffffff 02000000 03000000      .....
```

...

00000000004003f0 <printf@plt>:

```
4003f0: ff 25 22 0c 20 00    jmpq    *0x200c22(%rip) # 601018
```

<printf@GLIBC_2.2.5>

```
4003f6: 68 00 00 00 00      pushq   $0x0
4003fb: e9 e0 ff ff         jmpq    4003e0 <.plt>
```

Disassembly of section .text:

0000000000400400 <_start>:

```
400400: 31 ed              xor     %ebp,%ebp
```

....

00000000004004e7 <sum>:

```
4004e7: 55                push    %rbp      #①
4004e8: 48 89 e5          mov     %rsp,%rbp #②
4004eb: c7 45 fc 00 00 00 movl    $0x0,-0x4(%rbp) #③
4004f2: eb 1e            jmp     400512 <sum+0x2b>
4004f4: 8b 45 fc          mov     -0x4(%rbp),%eax
4004f7: 48 98            cltq
4004f9: 8b 14 85 30 10 60 mov     0x601030(,%rax,4),%edx
400500: 8b 05 3e 0b 20 00 mov     0x200b3e(%rip),%eax #601044 <val>
400506: 01 d0            add     %edx,%eax
400508: 89 05 36 0b 20 00 mov     %eax,0x200b36(%rip) #601044 <val>
40050e: 83 45 fc 01      addl    $0x1,-0x4(%rbp)
400512: 83 7d fc 03      cmpl    $0x3,-0x4(%rbp) #④
400516: 7e dc            jle     4004f4 <sum+0xd> #⑤
400518: 8b 05 26 0b 20 00 mov     0x200b26(%rip),%eax # 601044 <val>
```

```

40051e: 5d                pop    %rbp
40051f: c3                retq

```

0000000000400520 <main>:

```

400520: 55                push   %rbp
400521: 48 89 e5          mov     %rsp,%rbp
400524: 48 83 ec 10       sub     $0x10,%rsp
400528: 89 7d fc          mov     %edi,-0x4(%rbp)
40052b: 48 89 75 f0       mov     %rsi,-0x10(%rbp)
40052f: b8 00 00 00 00    mov     $0x0,%eax
400534: e8( ① )          callq   4004e7 <sum>
400539: 89 05( ② )       mov     %eax,■■■■■(%rip) #601044<val>
40053f: 8b 05( ③ )       mov     ■■■■■(%rip),%eax #601044<val>
400545: 89 c6            mov     %eax,%esi
400547: bf ( ④ )         mov     ■■■■■,%edi
40054c: b8 00 00 00 00    mov     $0x0,%eax
400551: e8 ( ⑤ )         callq   4003f0 <printf@plt>
400556: b8 00 00 00 00    mov     $0x0,%eax
40055b: c9              leaveq
40055c: c3              retq
40055d: 0f 1f 00        nopl    (%rax)

```

(1) 阅读的 `sum` 函数反汇编结果中带下划线的汇编代码（编号①-⑤），解释每行指令的功能和作用。

(2) 根据上述信息，链接程序从目标文件 `test.o` 和 `main.o` 生成可执行程序 `test`，对 `main` 函数中空格①--⑤所在语句所引用符号的重定位结果是什么？以 16 进制 4 字节数值填写这些空格，将机器指令补充完整。

(3) 在 `sum` 函数地址 `4004f9` 处的语句"`mov 0x601030(,%rax,4),%edx`"中，源操作数是什么类型、有效地址如何计算、对应 C 语言源程序中的什么量(或表达式)？其中，`rax` 数值对应 C 语言源程序中的哪个量(或表达式)？如何解释数字 4？

答：

(1)

1. `push %rbp`:

- 功能: 将当前函数的基指针（Base Pointer, `%rbp`）压入栈中。
- 作用: 保存调用函数（这里是 `main` 函数）的栈帧基指针，以便在 `sum` 函数返回时恢复调用者的栈帧。

2. `mov %rsp, %rbp`:

- 功能: 将栈指针（Stack Pointer, `%rsp`）的值复制到基指针（`%rbp`）。
- 作用: 设置当前函数（`sum`）的栈帧基指针。`%rbp` 现在指向当前栈帧的起始位置，用于访问局部变量和参数。结合 `push %rbp`，这两条指令完

成了新栈帧的建立。

3. `movl $0x0, -0x4(%rbp)`:
 - a) 功能: 将立即数 0 移动到 `%rbp` 减去 4 字节的内存位置 (即 `-0x4(%rbp)`)。
 - b) 作用: 初始化局部变量 `i` (在 C 代码中为 `int i`;) 为 0。 `-0x4(%rbp)` 是 `i` 的存储位置, 对应 C 源程序 `for (i=0; i<4; i++)` 中的 `i=0` 部分。
4. `cmpl $0x3, -0x4(%rbp)`:
 - a) 功能: 比较 `-0x4(%rbp)` (即变量 `i`) 的值与立即数 3。
 - b) 作用: 实现 C 源程序 `for` 循环的条件判断 `i<4`。由于 `i` 是整数, `i<4` 等价于 `i<= 3` (因为循环从 0 到 3)。比较结果设置条件标志 (如 `ZF`、`SF` 等), 用于后续条件跳转。
5. `jle 4004f4 <sum+0xd>`:
 - a) 功能: 如果比较结果满足“小于或等于” (Less or Equal, `jle`), 则跳转到地址 `4004f4` (循环体开始)。
 - b) 作用: 控制循环迭代。如果 `i<= 3` (条件为真), 跳转到循环体内部 (地址 `4004f4`, 对应 `val += a[i]`); 否则, 退出循环。`4004f4` 是循环体的第一条指令 (`mov -0x4(%rbp), %eax`), 加载 `i` 的值。结合 `cmpl $0x3, -0x4(%rbp)`, 这两条指令实现循环条件检查和分支。

(2)

1. 空格① (`callq sum`):
 - a) 指令地址: `0x400534` (`callq`), 偏移字段地址 `P = 0x400535` (`e8` 后 4 字节)
 - b) `RIP_next = 0x400539` (下一条指令)
 - c) 偏移 = `S_sum + A - RIP_next = 0x4004e7 + (-4) - 0x400539 = -0x52` (`0xFFFFFAE`)
 - d) 填充值: `AE FF FF FF` (小端: `FF FF FF AE` → 字节序列 `AE FF FF FF`)
2. 空格② (`mov to val`):
 - a) 指令地址: `0x400539` (`mov`), 偏移字段地址 `P = 0x40053b` (操作码 `89 05` 后)
 - b) `RIP_next = 0x40053f` (下一条指令)
 - c) 偏移 = `S_val + A - RIP_next = 0x601044 + (-4) - 0x40053f = 0x200B01`
 - d) 填充值: `01 0B 20 00` (小端: `00 20 0B 01` → 字节序列 `01 0B 20 00`)
3. 空格③ (`mov from val`):
 - a) 指令地址: `0x40053f` (`mov`), 偏移字段地址 `P = 0x400541` (操作码 `8b 05` 后)
 - b) `RIP_next = 0x400545` (下一条指令)
 - c) 偏移 = `S_val + A - RIP_next = 0x601044 + (-4) - 0x400545 = 0x200AFB`
 - d) 填充值: `FB 0A 20 00` (小端: `00 20 0A FB` → 字节序列 `FB 0A 20 00`)
4. 空格④ (`mov string address`):
 - a) 重定位类型 `R_X86_64_32`, 直接使用字符串地址 `0x4005e4` (`.rodata` 中 `"sum=%d\n"`)
 - b) 填充值: `E4 05 40 00` (小端: `00 40 05 E4` → 字节序列 `E4 05 40 00`)
5. 空格⑤ (`callq printf`):
 - a) 指令地址: `0x400551` (`callq`), 偏移字段地址 `P = 0x400552` (`e8` 后 4 字节)
 - b) `RIP_next = 0x400556` (下一条指令)
 - c) 偏移 = `S_printf + A - RIP_next = 0x4003f0 + (-4) - 0x400556 = -0x16A`

(0xFFFFFFFF96)

d) 填充值: 96 FE FF FF (小端: FF FF FE 96 → 字节序列 96 FE FF FF)

(3)

源操作数类型: 内存操作数 (从内存加载数据到寄存器%edx)。

有效地址计算: $0x601030 + \%rax * 4$

对应 C 语言源程序中的量: 数组 a 的第 i 个元素, 即 a[i]。C 代码中 val += a[i], a 是全局数组。

rax 数值对应 C 语言源程序中的量: %rax 的值对应循环变量 i (在符号扩展为 64 位后)。在 C 代码中, i 是 int 类型, cltq 指令 (4004f7) 将 %eax (32 位) 符号扩展为 %rax (64 位), 因此 %rax 存储 i 的值。

比例因子 4 表示数组元素的大小 (字节数)。因为 a 是 int 数组 (每个元素占 4 字节), 所以索引 i 需要乘以 4 来计算元素偏移。这对应 C 语言中的 sizeof(int)。